

KryoNet LibGDX Multiplayer POC

A practical multiplayer game networking example built for TDT4240 Software Architecture. This project demonstrates how to implement real-time client-server communication using KryoNet with LibGDX.

What This Project Does

This is a simple multiplayer game where players can:

- Connect to a central server
- Move around a shared 2D space
- See other players moving in real-time
- Send chat messages

It's intentionally minimal to serve as a starting point for your own multiplayer game.

Quick Start with VS Code

Prerequisites

Make sure you have installed:

- **Extension Pack for Java** (Microsoft) - search for it in Extensions (**Ctrl+Shift+X**)
- Java 11 or newer

Starting the Server (VS Code Debug)

1. Open the **Run and Debug** panel by clicking the play button with a bug icon in the left sidebar, or press **Ctrl+Shift+D**
2. At the top of the panel, click the dropdown menu (it may say "No Configurations" or show a previous config)
3. Select **"Run Server"**
4. Click the green play button, or press **F5**

The server starts in the integrated terminal. You should see:

```
=====
KryoNet Game Server POC
=====
Server started on TCP:27960 UDP:27961
```

Starting a Client (VS Code Debug)

With the server still running:

1. Go back to the **Run and Debug** panel (**Ctrl+Shift+D**)

2. Click the dropdown and select **"Run Desktop Client"**
3. Click the green play button

A LibGDX window opens showing your player as a green circle.

Running Multiple Clients

You can start additional clients:

1. Select **"Run Desktop Client 2"** from the dropdown
2. Click play

Each client appears as a different colored circle. You'll see them move in real-time.

Alternative: Using the Terminal

If the debug buttons don't appear, use the terminal instead:

Terminal 1 - Server:

```
.\gradlew.bat :server:run
```

Terminal 2 - Client (press **Ctrl+Shift+`** to open a new terminal):

```
.\gradlew.bat :desktop:run
```

Controls

Key	Action
WASD / Arrows	Move your player
Enter	Send a test chat message
P	Measure ping/latency
R	Reconnect to server

Project Structure

```
Gaming-Backend-POC/  
├── core/                                # Shared code (used by both client and server)  
│   └── com.example.network  
│       ├── NetworkClient.java          # Wraps KryoNet client  
│       ├── NetworkServer.java          # Wraps KryoNet server  
│       ├── NetworkConfig.java          # Port numbers, timeouts  
│       └── NetworkListener.java         # Interface for network events
```

```
|      └─ packets/                # Data classes for network messages
|      └─ server/                 # Standalone server application
|      └─ desktop/               # LibGDX desktop client
|      └─ android/               # Android client (tap to move)
```

Understanding the Networking

Why KryoNet?

We chose KryoNet for this project because:

1. **Built for games** - Created by the same developer as LibGDX, designed specifically for game networking needs
2. **TCP + UDP support** - Real-time games need both protocols:
 - **TCP** for messages that must arrive (login, chat, game events)
 - **UDP** for frequent updates where speed matters more than reliability (positions)
3. **Simple API** - Get multiplayer working quickly without deep networking knowledge
4. **Kryo serialization** - Your Java objects are automatically converted to compact bytes - much faster and smaller than JSON
5. **Works with LibGDX and Android** - No extra integration work needed

About KryoNet's Java Version

KryoNet was written for **Java 7** and the library hasn't been updated since 2018. However, this is perfectly acceptable for several reasons:

1. **Java is backwards compatible** - Code written for Java 7 runs fine on Java 11, 17, 21, or any newer version. Your project uses Java 11+, and KryoNet works without issues.
2. **Networking APIs haven't changed** - The Java NIO classes that KryoNet uses (channels, selectors, buffers) are stable and haven't been deprecated.
3. **It's feature-complete** - KryoNet does what it needs to do. Networking libraries don't need constant updates unless there are security issues.
4. **Battle-tested** - Many LibGDX games in production use KryoNet. Stability is more important than recency.

Can it be updated? The library is open source, so anyone could fork it and update the code style to use newer Java features (records, var, etc.). But functionally, there's nothing to fix - it works correctly as-is.

If you need a more actively maintained option for production, consider **Netty** (see comparison below).

How Network Communication Works

The client and server communicate through a simple request-response flow:

1. **Client connects** to the server via TCP (reliable connection)
2. **Client sends LoginRequest** with the player's name
3. **Server validates** and sends back LoginResponse (success/failure)
4. **Server broadcasts PlayerJoined** to all other connected clients
5. **During gameplay**, clients send PlayerPosition updates via UDP (fast, frequent)
6. **Server relays** position updates to all other clients
7. **Chat messages** go through TCP (must arrive reliably)
8. **When a client disconnects**, the server broadcasts PlayerLeft to others

The key insight: **the server is authoritative**. Clients send their intended actions, the server decides what actually happens, and broadcasts the result to everyone.

The Packet System

Packets are simple Java classes that hold data. KryoNet uses the Kryo library to automatically convert these objects to bytes for transmission.

```
// A packet is just a data class
public class PlayerPosition {
    public int playerId;
    public float x;
    public float y;
    public long timestamp;

    public PlayerPosition() {} // Required by Kryo!
}
```

Why the empty constructor? When Kryo receives bytes over the network, it needs to create an instance of the class first, then fill in the field values. Without a no-arg constructor, it can't create the instance.

Registration is critical. Every packet class must be registered in the same order on both client and server:

```
// In PacketRegistry.java - this runs on BOTH client and server
public static void register(Kryo kryo) {
    kryo.register(LoginRequest.class); // ID 0
    kryo.register(LoginResponse.class); // ID 1
    kryo.register(PlayerPosition.class); // ID 2
    // Order matters! Same order = same IDs
}
```

If the client registers `LoginRequest` first but the server registers `LoginResponse` first, they'll misinterpret each other's messages.

TCP vs UDP: When to Use Each

Use TCP for...	Use UDP for...
Login/logout	Position updates
Chat messages	Input states
Game events (damage, item pickup)	Animation states
Anything that <i>must</i> arrive	Anything sent many times per second

```
// TCP - guaranteed delivery, use for important messages
client.sendTCP(new ChatMessage("Hello!"));

// UDP - fast but may be lost, use for frequent updates
client.sendUDP(new PlayerPosition(playerId, x, y));
```

Programming Patterns Explained

This POC uses several software design patterns. Here's what they are and how we use them.

1. Observer Pattern

What it is: A pattern where an object (the "subject") maintains a list of dependents (the "observers") and notifies them automatically when its state changes. The subject doesn't need to know what the observers do with the information.

Why it's useful: It decouples the thing that produces events from the things that consume them. You can add new observers without changing the subject.

How we use it: The `NetworkClient` is the subject. It notifies all registered `NetworkListener` observers when network events occur (connected, disconnected, packet received).

```
// NetworkListener is the observer interface
public interface NetworkListener {
    void onConnected();
    void onDisconnected();
    void onReceived(Object packet);
}

// Your game screen becomes an observer
public class GameScreen implements NetworkListener {
    @Override
    public void onReceived(Object packet) {
        if (packet instanceof PlayerPosition) {
            // Update the game display
        }
    }
}
```

```
// Register as an observer - NetworkClient doesn't know or care what GameScreen
// does
client.addListener(gameScreen);

// You could add more observers for different purposes
client.addListener(soundEffects); // Play sounds on events
client.addListener(analytics);    // Track metrics
```

The `NetworkClient` just calls `listener.onReceived(packet)` for each listener. It doesn't know if listeners update graphics, play sounds, or log data.

2. Facade Pattern

What it is: A pattern that provides a simplified interface to a complex subsystem. The facade hides the complexity behind a clean API.

Why it's useful: Users of the facade don't need to understand the internals. If the internals change, only the facade needs updating.

How we use it: The `NetworkClient` and `NetworkServer` classes are facades over KryoNet. Your game code doesn't touch KryoNet directly.

```
// Without the facade, you'd have to do this:
Client kryoClient = new Client(16384, 4096);
kryoClient.getKryo().register(LoginRequest.class);
kryoClient.getKryo().register(LoginResponse.class);
// ... register 10 more classes
kryoClient.start();
kryoClient.connect(5000, "localhost", 27960, 27961);
kryoClient.addListener(new Listener() {
    public void received(Connection c, Object o) { ... }
});

// With our facade:
NetworkClient client = new NetworkClient(); // All setup done internally
client.connect("localhost");
client.addListener(myListener);
```

The facade handles buffer sizes, packet registration, threading, and error handling internally.

3. Registry Pattern

What it is: A pattern where a single class is responsible for registering and managing a collection of related items. It provides one place to configure something that would otherwise be scattered.

Why it's useful: Centralizes configuration. If you need to add a new packet type, there's one place to do it.

How we use it: The `PacketRegistry` class registers all packet types with Kryo in one place.

```
// PacketRegistry.java - the single source of truth for packet registration
public class PacketRegistry {
    public static void register(Kryo kryo) {
        // All packets registered here, in order
        kryo.register(LoginRequest.class);
        kryo.register(LoginResponse.class);
        kryo.register(PlayerPosition.class);
        kryo.register(ChatMessage.class);
        kryo.register(PlayerJoined.class);
        kryo.register(PlayerLeft.class);
        // ... add new packets at the end
    }
}

// Used in NetworkClient constructor
PacketRegistry.register(client.getKryo());

// Used in NetworkServer constructor
PacketRegistry.register(server.getKryo());
```

Without the registry, you'd have duplicate registration code in both `NetworkClient` and `NetworkServer`, and they might get out of sync.

4. Command Pattern

What it is: A pattern where requests are encapsulated as objects, allowing you to parameterize, queue, and log them. Each request becomes a self-contained "command" object.

Why it's useful: You can treat different types of requests uniformly, queue them, undo them, or log them.

How we use it: Each packet class is essentially a command. The server receives packets and dispatches them to handler methods based on type.

```
// Each packet type is a command object
public class LoginRequest {
    public String playerName;
    public String clientVersion;
}

public class ChatMessage {
    public String senderName;
    public String message;
}

// Server processes commands based on type
@Override
public void onReceived(PlayerConnection connection, Object packet) {
```

```

    if (packet instanceof LoginRequest) {
        handleLogin(connection, (LoginRequest) packet);
    } else if (packet instanceof PlayerPosition) {
        handlePosition(connection, (PlayerPosition) packet);
    } else if (packet instanceof ChatMessage) {
        handleChat(connection, (ChatMessage) packet);
    }
}

```

This makes it easy to add new packet types: create the class, register it, add a handler.

Framework Comparison for TDT4240

KryoNet

Pros	Cons
Made for LibGDX games	Java only
Very simple API	Last updated 2018 (but stable)
TCP + UDP built-in	No matchmaking or rooms
Fast Kryo serialization	Limited documentation
Works on Android	

Best for: Prototypes, LibGDX games, learning networking basics

Netty

Pros	Cons
Industry standard (used by Minecraft)	Steep learning curve
Extremely high performance	Verbose boilerplate code
Actively maintained	You handle serialization yourself
Excellent documentation	Overkill for simple games

Best for: Production servers, when you need maximum control

Socket.IO (Java Client)

Pros	Cons
Works with web browsers	TCP only (higher latency)
Built-in rooms and namespaces	Extra server dependency
Easy cross-platform support	Not ideal for real-time action

Pros

Cons

Good for chat features

Best for: Turn-based games, chat-heavy games, games with web clients

WebSockets

Pros

Cons

Browser compatible

TCP only

Standard protocol

Manual message handling

Works almost everywhere

No built-in serialization

Best for: Browser games, maximum compatibility

Recommendation for TDT4240

Use KryoNet if you're building a real-time LibGDX game and want to focus on gameplay rather than networking infrastructure. It's the fastest path to a working multiplayer prototype.

Consider Netty if you want to learn industry tools or need production-grade performance, but budget extra time for the learning curve.

Common Issues

"Address already in use"

This means another server instance is already running on the same port. Solutions:

1. **Check your terminals** - Look for existing "Run: ServerLauncher" terminals at the bottom of VS Code and close them
2. **Kill Java processes:**

```
taskkill /F /IM java.exe
```

3. **Restart VS Code** - This closes all terminals and their processes

VS Code doesn't show run configurations

1. Make sure **Extension Pack for Java** is installed (**Ctrl+Shift+X**, search for it)
2. Run **Ctrl+Shift+P** → "Java: Clean Java Language Server Workspace"
3. Run **Ctrl+Shift+P** → "Developer: Reload Window"

Client can't connect to server

1. Start the server first, wait for "Server started" message

2. Check Windows Firewall isn't blocking Java
3. Make sure you're connecting to `localhost` (or the correct IP for remote servers)

Packets not being received

1. Verify packet registration order is identical in `PacketRegistry.java` on both sides
2. Make sure all packets have a no-argument constructor
3. Check that packet fields are `public`

Extending This Project

Adding a New Packet Type

1. Create the packet class in `core/src/main/java/com/example/network/packets/`:

```
public class PlayerDamage {  
    public int targetId;  
    public int damage;  
    public int attackerId;  
  
    public PlayerDamage() {} // Required by Kryo!  
}
```

2. Register it in `PacketRegistry.java` (add at the end to maintain order):

```
kryo.register(PlayerDamage.class);
```

3. Handle it in your server/client:

```
if (packet instanceof PlayerDamage) {  
    PlayerDamage dmg = (PlayerDamage) packet;  
    players.get(dmg.targetId).takeDamage(dmg.damage);  
}
```

Deploying for LAN Play

1. Find your computer's IP: run `ipconfig` in terminal
2. Look for "IPv4 Address" (something like `192.168.1.100`)
3. Other players connect using that IP instead of `localhost`

Resources

- [KryoNet GitHub](#) - Source code and examples

- [Kryo Serialization](#) - How the serializer works
- [LibGDX Wiki](#) - LibGDX documentation
- [Game Networking Articles](#) - In-depth networking concepts